# End-to-End Big Data Log Analytics

## Using Hadoop, Spark, Kafka & MongoDB

**Name:** Deepak Devakumar S

**Reg. No:** RA2211047010076

**Course:** B.Tech – Artificial Intelligence

Made with GAMMA

# Abstract: Real-Time Log Analytics Pipeline

This project presents a robust, real-time big data pipeline designed for continuous log data ingestion and analytics. It seamlessly integrates open-source technologies like Hadoop, Spark, Kafka, and MongoDB to deliver live insights and error alerts through interactive dashboards.

## Integrated Pipeline

Combines Hadoop, Spark, Kafka, and MongoDB for comprehensive data flow.

## Streaming Analytics

Processes continuous log data for immediate insights and monitoring.

## Scalable & Fault-Tolerant

Engineered for high availability, automation, and seamless recovery.

## Interactive Dashboards

Streamlit provides live visualization, metrics, and error alerts.

Made with GAMMA

# Project Overview: Building a Unified Log Analytics System

Our goal is to construct an end-to-end pipeline for efficient log ingestion, processing, storage, and visualization. This system leverages industry-standard big data tools to handle the continuous stream of Apache web server logs.

## Platform

Developed and deployed on **Ubuntu WSL2**, providing a powerful Linux environment on Windows.

## Key Technologies

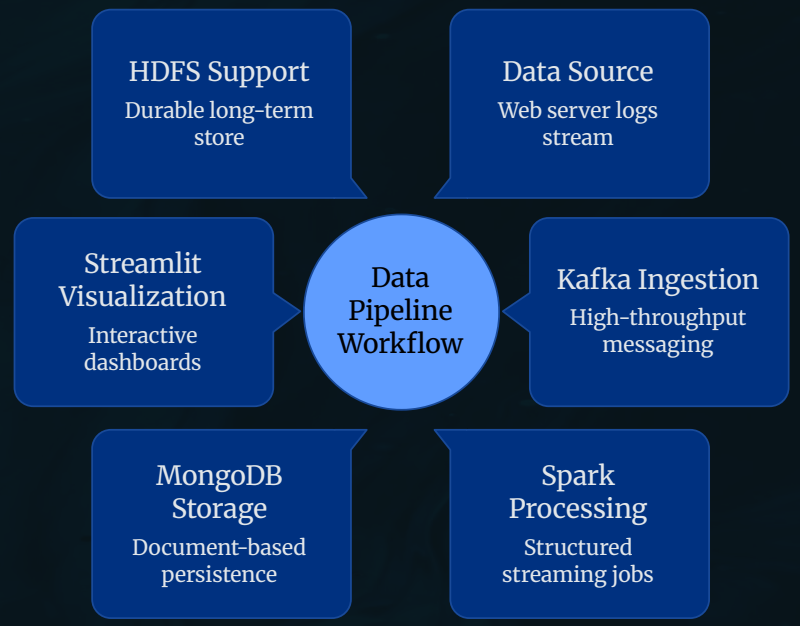
Hadoop (HDFS + YARN), Spark, Kafka, MongoDB, and Streamlit are integrated to create a cohesive system.

## Data Flow

Raw Apache web server logs serve as input, transformed into real-time analytics and error monitoring dashboards.

Made with GAMMA

# Workflow Architecture: Data's Journey Through the Pipeline




01

## Kafka Ingestion

Incoming web server logs are continuously streamed into **Kafka**, acting as a high-throughput messaging system.

02

## Spark Processing

**Spark Structured Streaming** consumes data from Kafka, performing real-time transformations and aggregations.

03

## MongoDB Persistence

Processed and aggregated results are efficiently stored in **MongoDB** for rapid retrieval and analysis.

04

## Streamlit Visualization

**Streamlit dashboards** dynamically display real-time metrics, providing actionable insights to users.

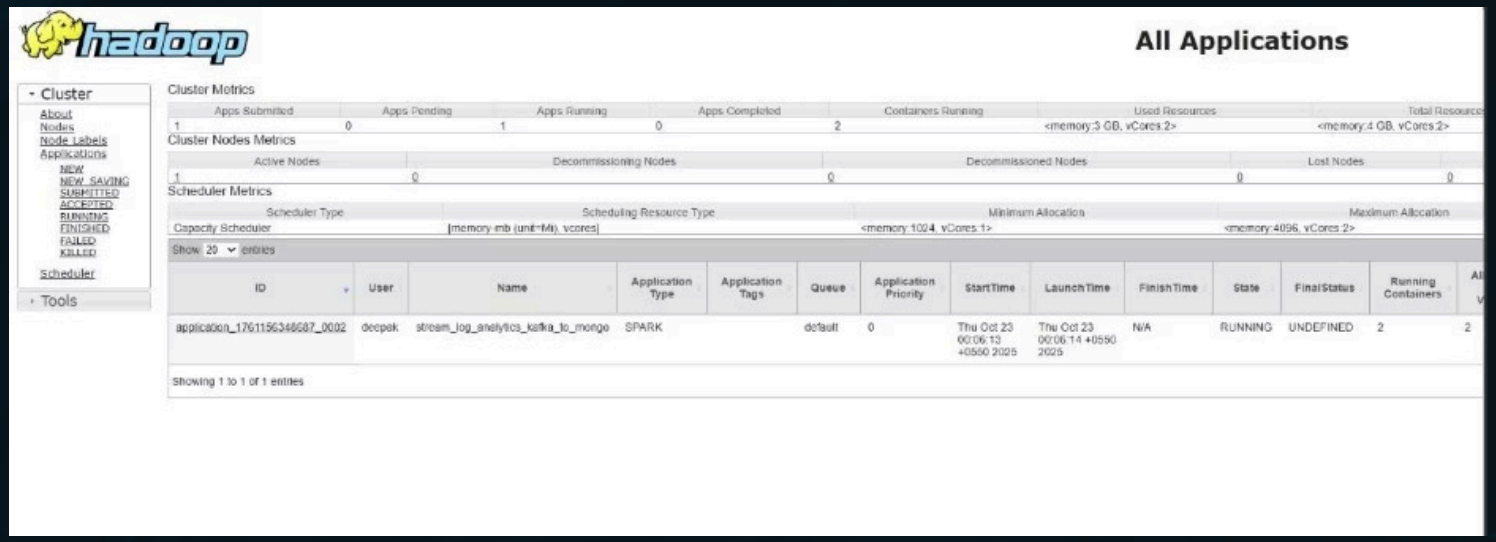
05

## HDFS Support

**HDFS** stores raw log data and Spark checkpoints, ensuring fault tolerance and data recovery.

Made with GAMMA

# Installation Steps: Setting Up the Big Data Environment




Detailed setup of each core component was crucial for building a stable and performant log analytics pipeline. Each tool was installed and verified to ensure seamless integration and functionality.

1. **Hadoop 3.3.6:** Installed and configured HDFS for distributed storage and YARN for resource management.
2. **Spark 3.5.2:** Verified installation with a SparkPi example, confirming computational capabilities.
3. **Kafka 3.7.0 (KRaft):** Set up Kafka in KRaft mode and created the dedicated topic `_web_logs_` for data ingestion.
4. **MongoDB 8.0:** Installed the NoSQL database and verified its operation using the `mongosh` client.
5. **Environment Configuration:** All necessary environment variables were meticulously configured to ensure proper inter-tool communication.

Made with GAMMA

# Real-Time Stream Processing with Spark

Spark Structured Streaming is the backbone of our real-time analytics, continuously transforming raw log data into valuable insights. Its fault-tolerance mechanisms ensure data integrity even in the face of disruptions.

## → Live Data Ingestion

Spark actively reads real-time log entries from the Kafka topic `_web_logs_`.

## → Data Transformation

Logs are efficiently tokenized, parsed, and aggregated, extracting critical information for analysis.

## → Continuous Output

Processed results are written to MongoDB in `foreachBatch()`, ensuring atomic and incremental updates.

## → Fault Tolerance

Spark checkpoints are stored in HDFS, enabling seamless recovery and preventing data loss during failures.

## → Update Mode Operation

The streaming job runs in "update mode" to provide incremental output updates, optimizing resource usage.

Made with GAMMA

# MongoDB Storage: Structured Log Data

MongoDB serves as the persistent storage layer for our processed log data, offering a flexible and scalable solution that supports rapid querying for dashboard visualization.

```
GNU nano 7.2                                    stream_logs.py
def process_batch(batch_df, epoch_id):
    if batch_df.isEmpty():
        return

    # === Aggregations on this micro-batch ===
    hits_by_hour = (batch_df.groupBy("date","hour")
                    .agg(F.count("*").alias("hits"),
                         F.sum(F.when(F.col("status") >= 500, 1).otherwise(0)).alias("errors"))
                    .orderBy("date","hour"))

    top_urls = (batch_df.groupBy("date","url")
                .agg(F.count("*").alias("hits"))
                .orderBy(F.desc("hits"))
                .limit(1000))

    status_counts = (batch_df.groupBy("date","status")
                     .agg(F.count("*").alias("count"))
                     .orderBy("date","status"))

    # write each table to MongoDB (append streaming results)
    write_mongo(hits_by_hour, "hits_by_hour", mode="append")
    write_mongo(top_urls, "top_urls", mode="append")
    write_mongo(status_counts, "status_counts", mode="append")
```

### Database Setup

Dedicated database named `bdproj` created for storing all processed log analytics.

### Key Collections

Multiple collections like `agg_results`, `hits_by_hour`, `top_urls`, and `status_counts` categorize different metrics.

### Data Structure

Each document stores aggregated log data alongside timestamps, facilitating time-series analysis.

### Optimized for Speed

MongoDB's design ensures fast query performance, crucial for real-time dashboard updates.

### Data Verification

Data integrity and presence verified using commands such as `db.agg_results.find().limit(5)`.

Made with GAMMA

# Automation Scripts: Streamlining Operations

To simplify the complex process of managing multiple big data services, custom automation scripts were developed. These scripts ensure consistent startup and shutdown procedures, enhancing reproducibility and maintainability.

### `start_all.sh`

This script orchestrates the sequential startup of HDFS, YARN, Kafka, Spark, and the log data producer.

### `stop_all.sh`

Gracefully terminates all running services and performs necessary cleanup by removing PID files.

### Lifecycle Management

These scripts automate the entire project lifecycle, from deployment to teardown.

### Reproducibility

Ensures a clean and consistent environment setup, making it easy to reproduce results or redeploy the system.

Made with GAMMA

```
. .venv/bin/activate
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ start_all.sh
start_all.sh: command not found
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$ ./start_all.sh
=== STARTING BIG DATA PIPELINE ===
→ Starting HDFS...
→ Starting YARN...
→ Verifying Hadoop processes...
1187 SecondaryNameNode
757 NameNode
904 DataNode
1433 ResourceManager
1580 NodeManager
→ Starting Kafka (KRaft config)…
→ Locating spark-submit...
  Using: /home/deepak/spark-hdfs-mongo/.venv/bin/spark-submit
→ Launching Spark streaming job (Kafka → Mongo)…
  Spark job started (log: /home/deepak/spark-hdfs-mongo/logs/spark_stream.log)
→ Starting Kafka log producer (live timestamps)…
→ Starting Streamlit dashboards…
=== PIPELINE READY ===
✅ Spark streaming job → MongoDB
✅ Kafka producer active
✅ Dashboards:
   → http://localhost:8501 (Alerts)
   → http://localhost:8502 (Analytics)
Logs → /home/deepak/spark-hdfs-mongo/logs
(.venv) deepak@SENTRY11:~/spark-hdfs-mongo$
```

Made with GAMMA

# Results and Output: Live Insights in Action

The integrated pipeline successfully demonstrates continuous log processing and real-time visualization, proving its efficacy in a big data environment.

## YARN Integration

Spark streaming jobs actively managed within the YARN ResourceManager UI.

## MongoDB Data Storage

Aggregated log data and analytics results consistently stored in MongoDB collections.

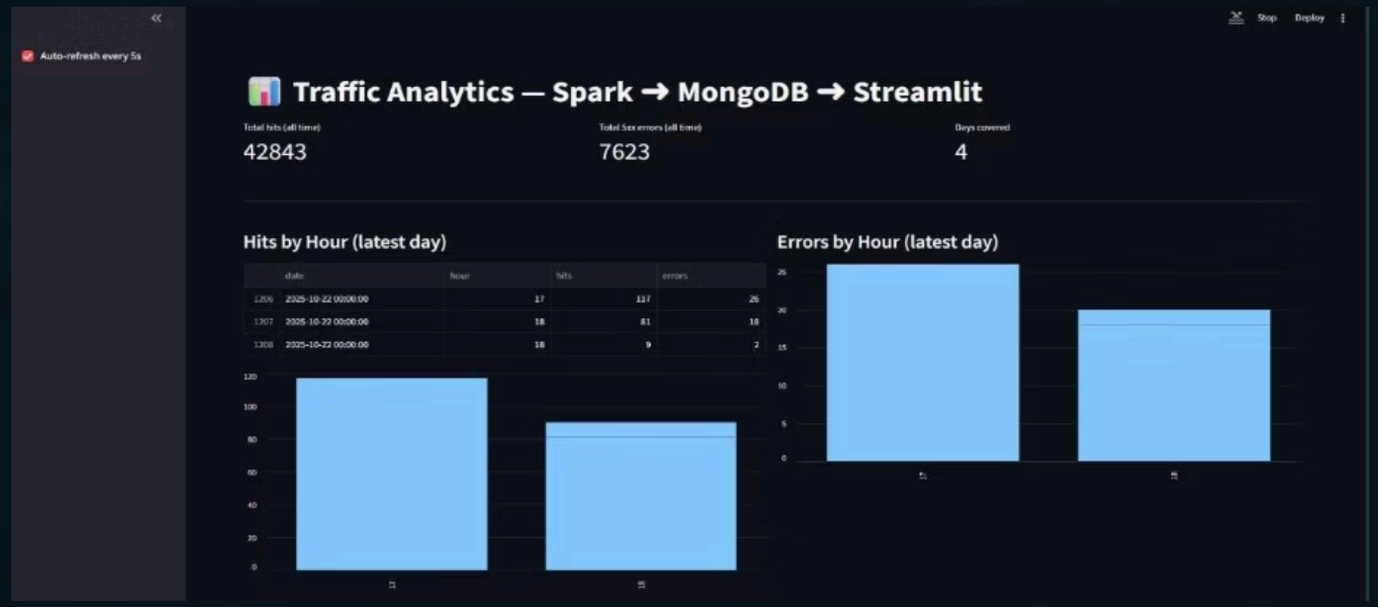
## Streamlit Dashboards

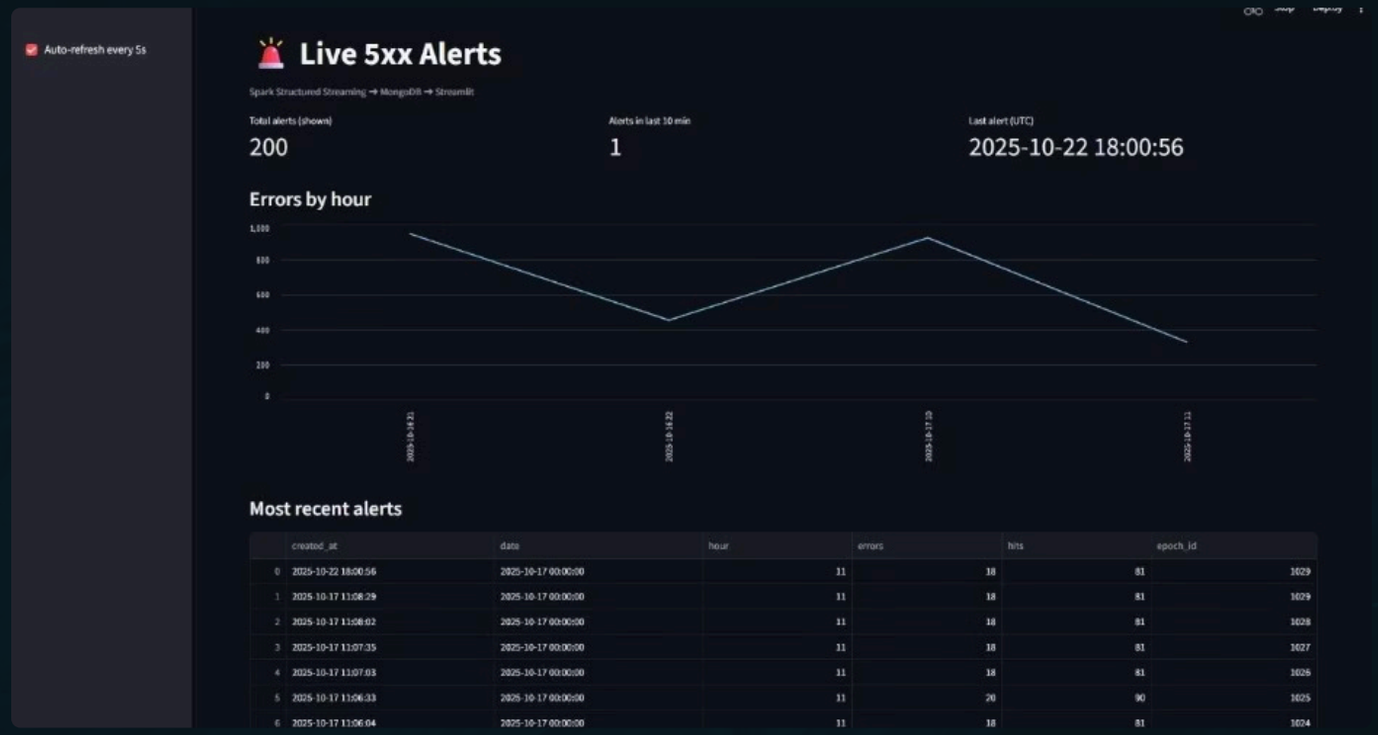
Dynamic dashboards visualize live traffic, user behavior, and critical error rates.

## Autonomous Operation

The entire pipeline operates continuously and autonomously.

Made with GAMMA




Made with GAMMA

# Conclusion and Future Scope

This project successfully demonstrates a robust, end-to-end big data pipeline for real-time log analytics. It integrates core components—Hadoop, Spark, Kafka, and MongoDB—to deliver scalable processing and dynamic visualization.

### Successful Integration

Achieved seamless integration of Hadoop, Spark, Kafka, and MongoDB.

### Real-Time Processing

Demonstrated effective real-time data streaming and analytics.

### Dynamic Visualization

Enabled live insights through interactive Streamlit dashboards.

### Containerized Deployment

Transition to Docker/Kubernetes for enhanced scalability and portability.

### Anomaly Detection

Implement ML-based anomaly detection for proactive error identification.

### Multi-Node Scaling

Expand to a multi-node Hadoop cluster for handling larger data volumes.

Made with GAMMA